

Machine Learning Assignment Week 2

Dataset Header: #1-1-1

Stephen Haskins: 21365534

October 15, 2025

Introduction

This assignment implements logistic regression and linear support vector machines for a data classification task. The dataset's non-linear class separation highlights the shortcomings of linear models and the importance of feature engineering. The impact of the regularisation parameter on the SVM model is also investigated, with compact visualisations demonstrating results.

A) Logistic Regression

i) Visualise the data

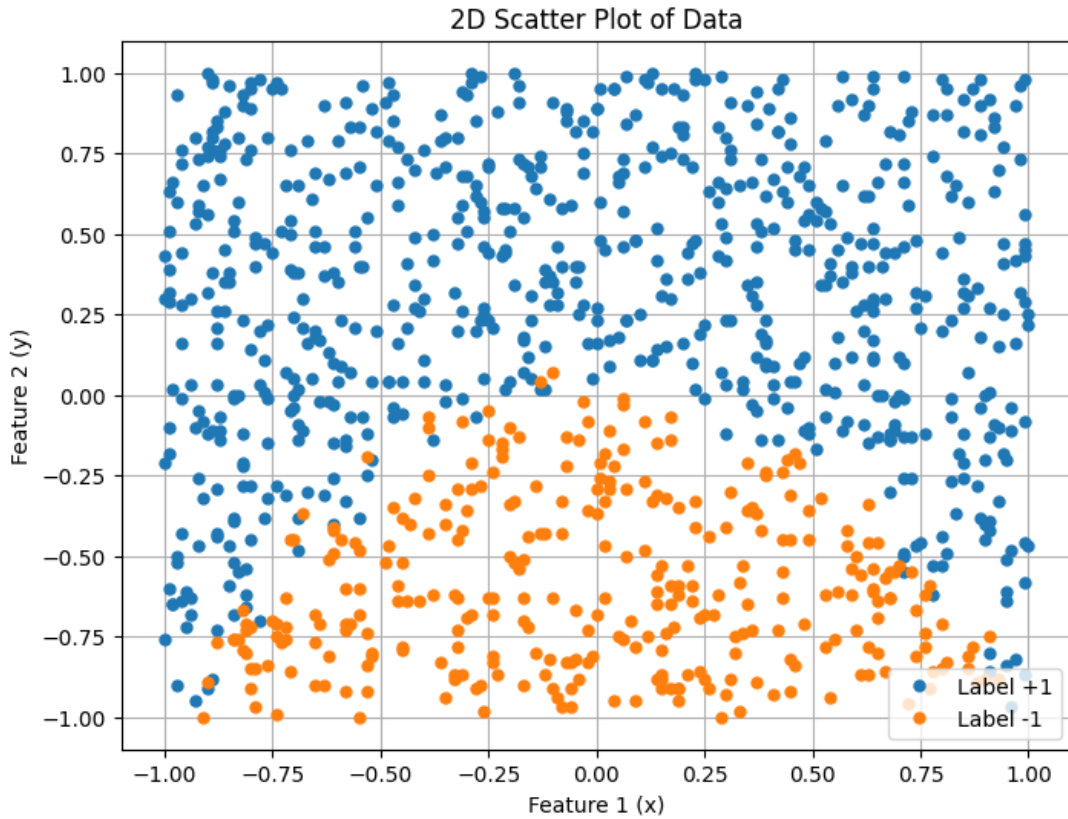


Figure 1: 2D Scatter Plot of Ground Truth

Figure 1 shows a 2D scatter plot of the provided data, with blue and orange point markers to represent the two labels (+1, -1) respectively. The data itself contains two features (x, y) as well as two labels (+1, -1) and is bounded in the feature space $[-1, 1] \times [-1, 1]$. There is a relatively even dispersion of data points across the feature space with classes clustered via a clear non-linear decision boundary.

ii) Train the Logistic Regression model

The logistic regression model was implemented using sklearn’s LogisticRegression class with its default parameters ([1]). Note L2 regularisation is enabled by default. L2 regularisation is a technique used to reduce model complexity and prevent overfitting by penalising large weights. This logistic regression model predicts the probability of the label $y \in -1, 1$ for a given feature vector $\mathbf{x}$. The model estimates its parameters by maximising the log-likelihood of the data, thus optimising the coefficients to best separate the classes in the log-odds space. Predictions are generated using the sigmoid function, and classification is performed by thresholding the predictions at 0.5.

The model parameters are as follows:

Parameter	Value	Description
w_0	1.8001	Intercept / bias
w_1	-0.2096	Coefficient for x
w_2	5.0191	Coefficient for y

Table 1: Logistic regression model parameters.

This gives us the model:

$$P(y = 1 | x, y) = \sigma(1.8001 - 0.2096x + 5.0191y) \quad (1)$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

and $y \in \{-1, 1\}$.

The logistic regression model’s coefficient’s reveal each feature’s importance and directional influence on classification predictions.

Feature 2 (y) displays the most influence on predictions with a coefficient magnitude of 5.0191, compared to Feature 1 (x) with magnitude 0.2096. This means changes in Feature 2 will have a much larger impact on the model’s predictions, which, looking at the data visualisation in Figure 1, makes sense. There is clear vertical class separation. An optimised horizontal decision boundary would result in far superior class separation compared to an optimised vertical decision boundary.

The coefficient signs indicate the directional relationship between features and class predictions. Feature 2 with a positive coefficient indicates a positive correlation with probability of predicting the +1 class. The data’s previously mentioned vertical class separation means that as Feature 2 increases, the probability of Label +1 increases, which perfectly matches the positive coefficient. On the other hand, feature 1 with a negative coefficient decreases the probability of predicting the +1 class.

The intercept value of 1.8001 suggests a bias toward predicting class +1 when both features equal zero. This baseline bias is expected and aligns with the observed data distribution in Figure 1, where the majority of data points belong to class +1.

iii) Predict target values

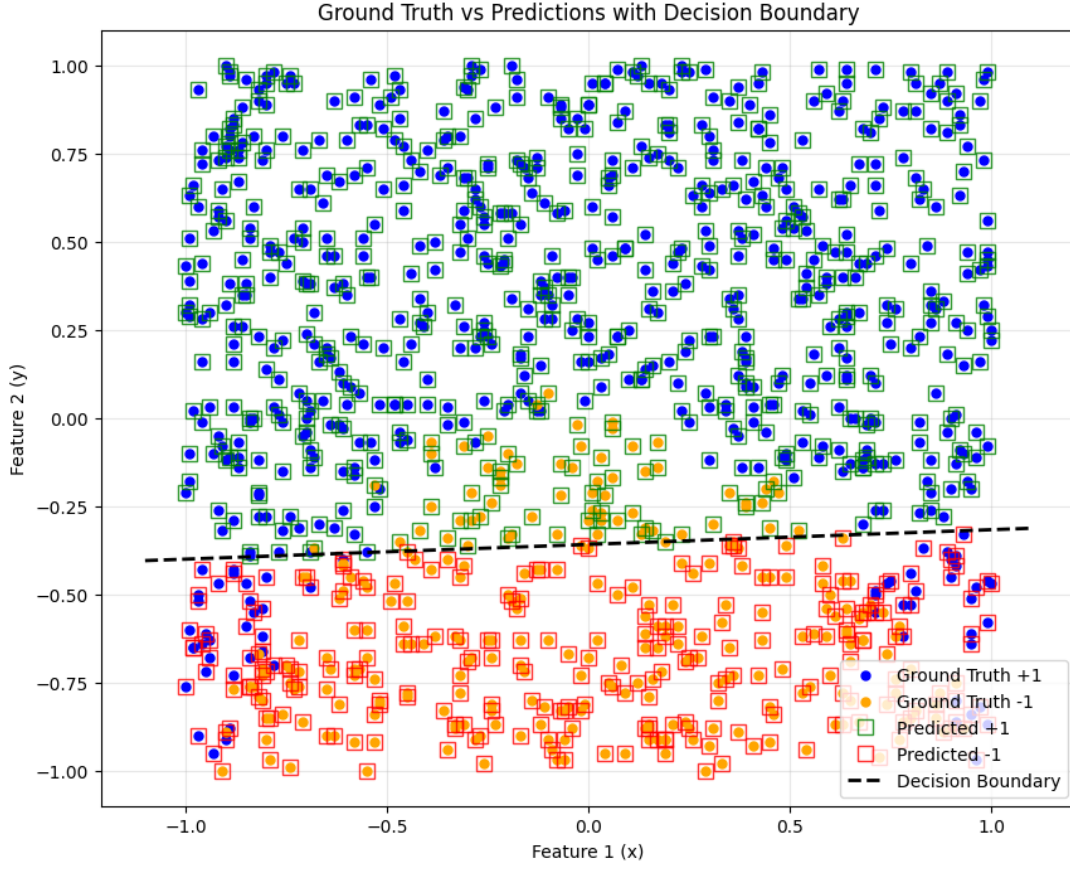


Figure 2: Logistic Regression Predictions

The predicted target values for the trained model are shown in Figure 2 using coloured green and red box markers for +1 and -1 predictions respectively. The decision boundary is depicted by the black dashed line and was calculated as follows:

The decision boundary is defined as the set of points where the model is maximally uncertain, i.e., when $P(y = 1 | x) = 0.5$. From equations (1) and (2), it can be deduced that this is equivalent to the case where the argument of the sigmoid function is zero, i.e., $z = 0$.

$$1.8001 - 0.2096x + 5.0191y = 0 \quad (3)$$

Solving for y in terms of x gives:

$$y = \frac{-1.8001 + 0.2096x}{5.0191} \quad (4)$$

A range of x values were substituted into Equation (4) to generate the decision boundary.

iv) Discuss the results

The performance metrics in Table 2 were calculated using the sklearn.metrics module [2] and are as follows:

Metric	Value
Accuracy	0.8719
Precision	0.9045
Recall	0.9098
F1-Score	0.9071

Table 2: Performance metrics for the logistic regression model.

The logistic regression model in Figure 2 achieves reasonable classification performance given the model’s linear assumption. There is a moderate amount of misclassified data points near the decision boundary. The linear decision boundary cannot capture the non-linear nature of the underlying data, indicating underfitting. This is reflected in the performance metrics in Table 2, with an overall accuracy of 87.2% and balanced precision/recall values. These results indicate strong but not perfect classification performance, consistent with the visible boundary misclassifications.

B) Support Vector Machines

i) Train an SVM model

Support vector machines find the hyperplane that maximises the separation between classes in the feature space. The SVM optimisation objective is to maximise the margin, ie the distance between the decision boundary and the nearest training points (support vectors) from each class. This is achieved by minimising $\frac{1}{2}\|w\|^2$.

Just like the logistic regression model, the sklearn library and it’s LinearSVC class was used in this implementation [3]. The LinearSVC class by default uses l2 regularisation, and takes a penalty parameter C.

The strength of this l2 regularisation is inversely proportional to C, meaning smaller C values result in stronger regularisation, while larger C values result in weaker regularisation. A wide range of C values were used in order to effectively visualise the impact of the regularisation strength on the model. The following table shows the model parameters for different values of C.

C	Intercept $[w_0]$	Coefficients $[w_1, w_2]$	Model
0.0001	0.0622	$[-0.0036, 0.0739]$	$f(x_1, x_2) = 0.0622 - 0.0036x_1 + 0.0739x_2$
0.001	0.2424	$[-0.0180, 0.4671]$	$f(x_1, x_2) = 0.2424 - 0.0180x_1 + 0.4671x_2$
1	0.6448	$[-0.0734, 1.8435]$	$f(x_1, x_2) = 0.6448 - 0.0734x_1 + 1.8435x_2$
100	0.6522	$[-0.0743, 1.8623]$	$f(x_1, x_2) = 0.6522 - 0.0743x_1 + 1.8623x_2$

Table 3: SVM model parameters for different values of C

ii) Predict target values

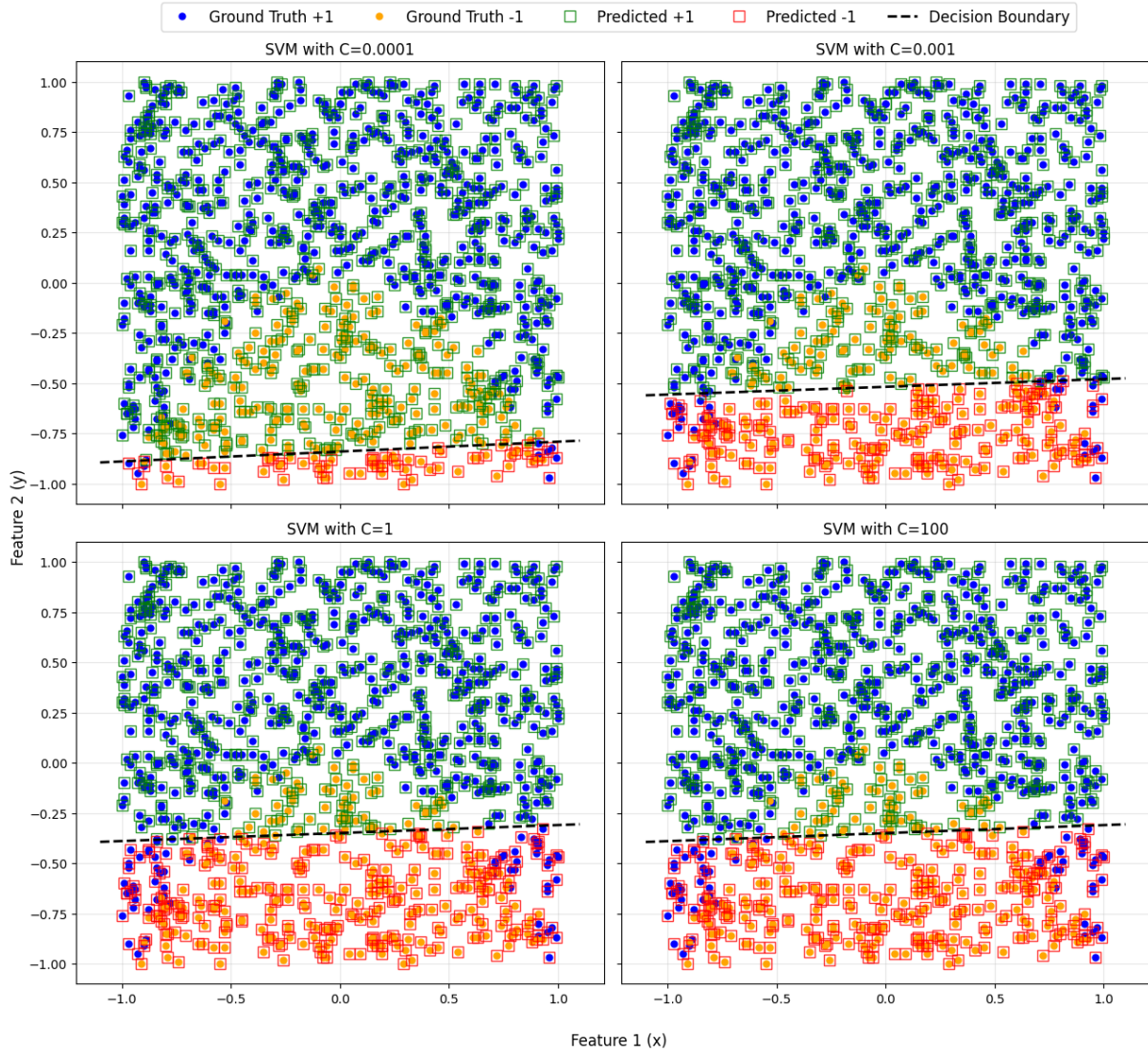


Figure 3: SVM Predictions for different values of C

Figure 3 shows the SVM predictions for different values of C in a similar fashion to previously. The ground truth is represented by blue and orange point markers, and predictions are overlaid via square markers. The decision boundary is depicted by the black dashed line and was calculated as before.

iii) Discuss the impact of C

C	Accuracy	Precision	Recall	F1-Score
0.0001	0.7427	0.7322	0.9869	0.8407
0.0010	0.8519	0.8523	0.9491	0.8981
1.0000	0.8699	0.9054	0.9054	0.9054
100.0000	0.8699	0.9054	0.9054	0.9054

Table 4: Comparison of SVM performance metrics for different values of C .

Table 5 shows the model parameters for different values of C . The margin is calculated as $\frac{2}{\|w\|}$.

C	Intercept w_0	Coefficients $[w_1, w_2]$	Margin
0.0001	0.0622	$[-0.0036, 0.0739]$	27.01
0.0010	0.2424	$[-0.0180, 0.4671]$	4.28
1.0000	0.6448	$[-0.0734, 1.8435]$	1.08
100.0000	0.6522	$[-0.0743, 1.8623]$	1.07

Table 5: Comparison of SVM model parameters & margins across different values of C .

At low values of C (0.0001 & 0.001), the model exhibits underfitting characteristics, producing a decision boundary heavily favouring the majority class. This is reflected in table 5 where coefficient parameters are extremely small, leading to a very large margin.

Such strong regularisation can induce a bias toward the majority class as the simplest solution that minimises overall error is to bias heavily toward the majority class. This behaviour is seen in table 4. For $C=0.0001$, recall is extremely high (98.7%) while precision is low (73.2%), indicating that the classifier is predicting the +1 class for most points.

Between $C=1$ and $C=100$, the coefficient magnitudes grow substantially and the margin shrinks (from 27.0 to 1.08), indicating a tighter fit to the training data. The intercept also increases, shifting the decision boundary upwards to better align with the vertical class separation. The accuracy, precision, recall, and F1-score in 4 are identical for both values, indicating that the model has reached a stable margin configuration where further reductions in regularisation have negligible effect. Visually, misclassifications decrease considerably, indicating improved training accuracy.

While increasing C reduces regularisation and can improve the fit to training data, excessively large C values risk overfitting, as the model may start to fit noise rather than the true decision boundary.

This demonstrates the practical trade-off: too much regularisation (low C) produces overly simple models that fail to learn the underlying pattern, while moderate to high C values may lead to overfitting and reduced performance on unseen data. The ideal C values can be determined via cross-validation, and will be done so in the following assignment.

iv) Compare the SVM model to Logistic Regression

The performance analysis reveals a fundamental limitation of the two linear models. Both Logistic Regression and the SVM (with $C=1$) converged on a similar linear decision boundary, resulting in nearly identical performance metrics with roughly 87% accuracy (Tables 2 and 4). This is not a coincidence but rather evidence that both models are constrained by their linear assumptions. Their inability to exceed this performance ceiling, despite their different internal mechanics, strongly indicates that the underlying data pattern is non-linear. The consistent pattern of misclassifications visible in Figures 2 and 3 further confirms this.

Comparing the model parameters, both approaches assign greater weight to Feature 2 (y) than to Feature 1 (x), reflecting the dominant vertical separation in the data. Logistic regression produces a larger coefficient for Feature 2 (5.02) than the SVM with $C=1$ (1.84). This is expected behaviour. Logistic regression maximises the log-likelihood, with contributions from all data points, including those far from the decision boundary. This cumulative effect can lead to larger coefficient magnitudes, as the model pushes probabilities toward 0 or 1 across the dataset. In contrast, the SVM solution depends only on support vectors — the data points closest to the

decision boundary. Points far from the margin do not affect the solution, which often leads to smaller coefficient magnitudes compared to logistic regression.

C) Feature Augmentation

i) Create additional features and train a Logistic Regression model

This section expands upon Part A by adding two additional features representing the squares of the original features. This results in the feature vector $[x_1, x_2, x_1^2, x_2^2]$. This feature engineering can help capture the non-linear nature of the data by transforming the feature space to a higher dimension. A logistic regression model was then trained using these new features. The feature coefficients are as follows:

Feature	Coefficient	Effect on Prediction
x	-0.1500	Decreases prediction
y	6.9671	Increases prediction
x^2	6.4446	Increases prediction
y^2	-0.5419	Decreases prediction

Table 6: Feature coefficients and their effects in the poly LR model.

ii) Predict target values

In this higher-dimensional space, the logistic regression model can still find a linear decision boundary, but when projected back to the original 2D scatter plot, this becomes a curved, non-linear boundary. This can be seen in Figure 4 below.

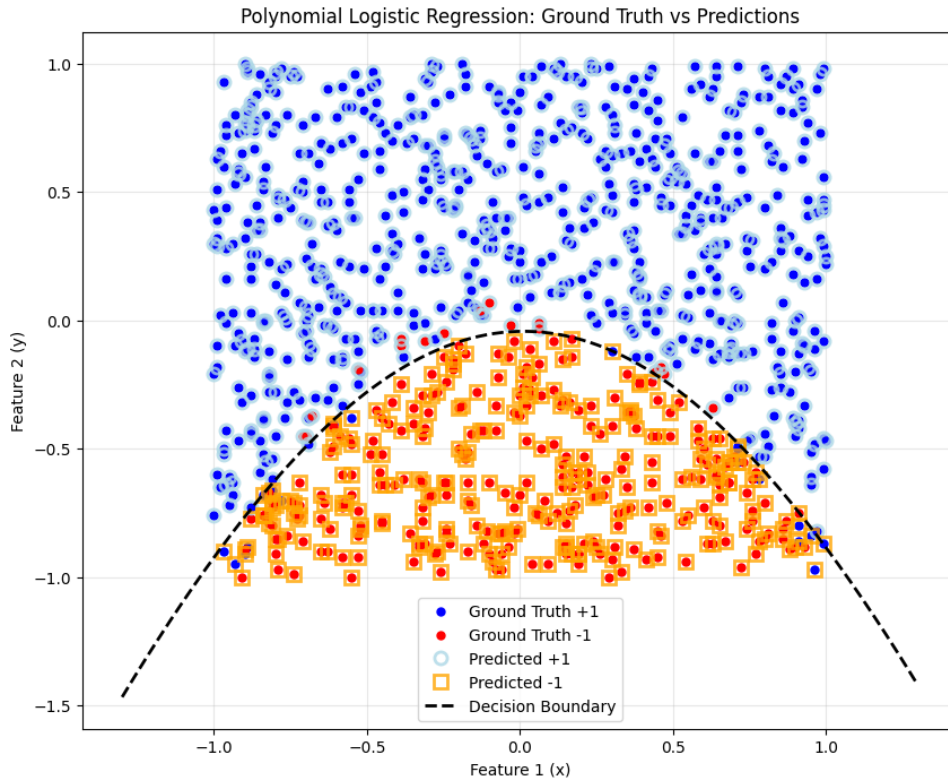


Figure 4: Logistic Regression Predictions with Polynomial Feature Augmentation

iii) Compare against baseline

A simple baseline classifier was created to demonstrate the logistic regression model actually learned something useful. This baseline predictor is a simple majority class classifier, always predicting the most frequent class in the dataset. The polynomial logistic regression model substantially outperforms the baseline, demonstrating that it has learned meaningful discriminatory patterns from the data, as shown in Table 7 below.

Metric	Baseline	Poly LR	Improvement
Accuracy	0.6877	0.9640	+0.2763
Precision	0.6877	0.9752	+0.2875
Recall	1.0000	0.9723	-0.0277
F1	0.8149	0.9738	+0.1588

Table 7: Performance comparison between baseline and polynomial logistic regression.

The baseline model achieves an accuracy of 68.77%, which reveals that the dataset is imbalanced, with approximately 69% positive samples and 31% negative samples. In contrast, the polynomial logistic regression model achieves an accuracy of 96.40%, representing a substantial improvement of 27.63%. This significant gain indicates that the model is genuinely learning the underlying patterns in the data, rather than simply exploiting the dataset’s class imbalance.

The improvement in precision is particularly noteworthy. The baseline’s precision of 68.77% is equal to its accuracy because it always predicts the positive class and never predicts class -1 , resulting in 31% of its positive predictions being false positives. The polynomial model, however, achieves a precision of 97.52%, an increase of 28.75 percentage points. Recall decreases slightly from 100% to 97.23% (a reduction of 2.77%). This shows the model has learned to distinguish between the two classes rather than simply predicting positive for all samples.

The F1-score, which provides a balanced measure of both precision and recall, offers a more comprehensive view of model performance than accuracy alone. The polynomial model achieves an F1-score of 97.38%, 15.88% higher than the baseline’s 81.49%. The baseline’s F1-score is artificially inflated by its perfect recall despite poor precision, whereas the polynomial model demonstrates strong performance across both metrics. This highlights how considering a single metric alone can be misleading.

By incorporating polynomial features, the model is able to capture the non-linear decision boundary present in the data. This results in a classifier that meaningfully distinguishes between classes, rather than simply predicting the majority class. The model’s ability to maintain high recall (97.23%) while dramatically improving precision (97.52%) demonstrates that it has effectively learned the underlying structure of the data.

iv) Bonus

Unlike the simple linear models where the decision boundary was a straight line, this polynomial logistic regression uses 4 features

$$x, y, x^2, y^2$$

The decision boundary occurs where the model’s prediction probability equals 0.5. From the coefficient values from Table (4), this gives the following equation:

$$0.298 - 0.1500x_1 + 6.9671x_2 + 6.4446x_1^2 - 0.5419x_2^2 = 0 \quad (5)$$

This equation can be rearranged to create a quadratic equation in terms of x_2 :

$$-0.5419x_2^2 + 6.9671x_2 + 0.298 - 0.1500x_1 = 0 \quad (6)$$

This equation can be solved for x_2 using the quadratic formula:

$$x_2 = \frac{-6.9671 \pm \sqrt{6.9671^2 - 4 \cdot -0.5419 \cdot (0.298 - 0.1500x_1)}}{2 \cdot -0.5419} \quad (7)$$

giving two solutions, only one of which is valid and plotted in Figure 4.

Conclusion

This assignment demonstrates how linear models (logistic regression and SVM) achieve moderate performance (87% accuracy) on non-linearly separable data, but suffer from underfitting due to their linear decision boundaries. Feature augmentation through polynomial terms dramatically improves performance to 96.4% accuracy by enabling the model to capture the underlying non-linear decision boundary in a higher-dimensional space.

References

- [1] Sklearn logistic regression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html, 2025.
- [2] Sklearn metrics. https://scikit-learn.org/stable/modules/model_evaluation.html, 2025.
- [3] Sklearn linear svm. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>, 2025.

Appendix: Code

The following is a pdf of the Jupyter Notebook used in this assignment.

id:1-1-1

Imports

```
In [24]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

The data file consists of three columns of data (plus the first header line). The first two columns are input features and the third column is the real-valued target value.

```
In [25]: df = pd.read_csv("data.csv")
print(df.head(10))
X1 = df.iloc[:, 0]
X2 = df.iloc[:, 1]
X = np.column_stack((X1, X2))
y = df.iloc[:, 2]
```

	x	y	label
0	0.64	0.30	1
1	-0.01	0.82	1
2	-0.82	0.23	1
3	-0.71	0.39	1
4	-0.47	0.28	1
5	0.49	0.45	1
6	0.41	-0.01	1
7	0.92	0.63	1
8	-0.10	-0.63	-1
9	0.05	0.66	1

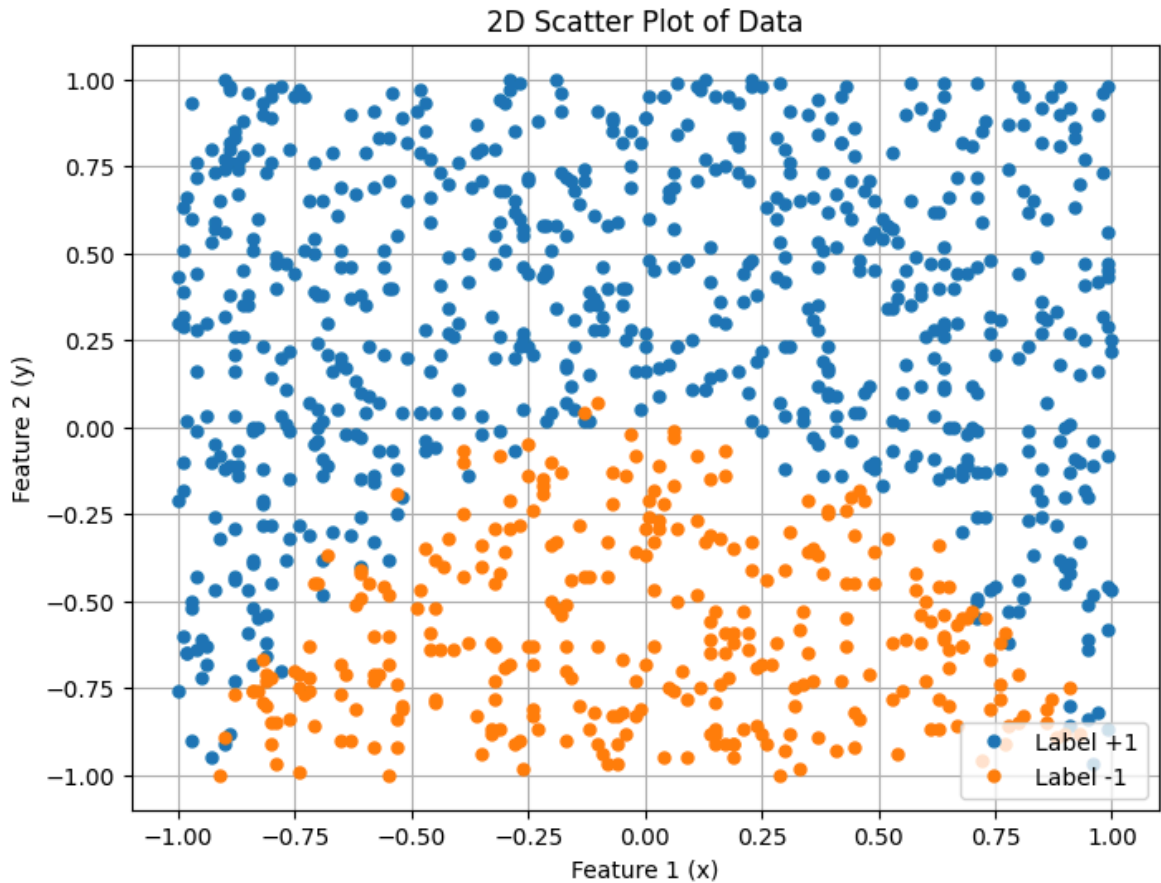
2D Scatter Plot

```
In [26]: plt.figure(figsize=(8, 6))

# Use boolean mask to filter rows X[y==n, :]
X_pos = X[y==1, :]
X_neg = X[y==-1, :]

plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Label +1', markersize=10)
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Label -1', markersize=10)

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('2D Scatter Plot of Data')
plt.legend()
plt.grid(True)
plt.show()
```



Logistic Regression

ii) Use sklearn to train a logistic regression classifier on the data. Give the logistic regression model for predictions and report the parameter values of the trained model. Discuss e.g. which feature has most influence on the prediction, which features cause the prediction to increase and which to decrease

```
In [27]: # train the model
lr = LogisticRegression()
lr.fit(X, y)
```

```
Out[27]: ▼ LogisticRegression ⓘ ?
```

```
► Parameters
```

```
In [28]: lr.get_params()
```

```
Out[28]: {'C': 1.0,
          'class_weight': None,
          'dual': False,
          'fit_intercept': True,
          'intercept_scaling': 1,
          'l1_ratio': None,
          'max_iter': 100,
          'multi_class': 'deprecated',
          'n_jobs': None,
          'penalty': 'l2',
          'random_state': None,
          'solver': 'lbfgs',
          'tol': 0.0001,
          'verbose': 0,
          'warm_start': False}
```

```
In [29]: # Get the model parameters
print("Intercept:", lr.intercept_[0])
print("Feature 1 coef:", lr.coef_[0][0])
print("Feature 2 coef:", lr.coef_[0][1])
```

```
Intercept: 1.8001096150590803
Feature 1 coef: -0.2096025064278186
Feature 2 coef: 5.019054062423062
```

iii) Now use the trained logistic regression classifier to predict the target values in the training data. Add these predictions to the 2D plot you generated in part (i), using a different marker and colour so that the training data and the predictions can be distinguished. Show the decision boundary of the logistic regression classifier as a line on the plot (you'll need to use the parameter values of the trained model to figure out what line this should be - explain how you obtain it).

```
In [30]: y_pred = lr.predict(X)
y_pred_proba = lr.predict_proba(X)

print("Predictions:", y_pred[:10])
print("Prediction probabilities:", y_pred_proba[:10])
```

```
Predictions: [ 1  1  1  1  1  1  1  1 -1  1]
Prediction probabilities: [[0.04024547 0.95975453]
 [0.00268381 0.99731619]
 [0.04203269 0.95796731]
 [0.01971703 0.98028297]
 [0.03543566 0.96456434]
 [0.01878047 0.98121953]
 [0.15922818 0.84077182]
 [0.00841508 0.99158492]
 [0.79264353 0.20735647]
 [0.00604653 0.99395347]]
```

```
In [31]: X_pos_pred = X[y_pred == 1, :]
X_neg_pred = X[y_pred == -1, :]
```

```

In [67]: plt.figure(figsize=(10, 8))

X_pos = X[y==1, :]
X_neg = X[y==-1, :]

# plot GT
plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', markersize=10)
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', markersize=10)

# plot predictions
plt.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 's', label='Predicted +1', markersize=10,
         color='green', alpha=0.8, fillstyle='none', markeredgewidth=1.5)

plt.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1', markersize=10,
         color='red', alpha=0.8, fillstyle='none', markeredgewidth=1.5)

# decision boundary
# Step 1: Extract model parameters
intercept = lr.intercept_[0] # bias term
coef = lr.coef_[0]          # [coef_x1, coef_x2]

# Step 2: Decision boundary occurs when  $P(y=1|x) = 0.5$ 
# happens when  $z = 0$ 
# Step 3: Solve for x2 in terms of x1

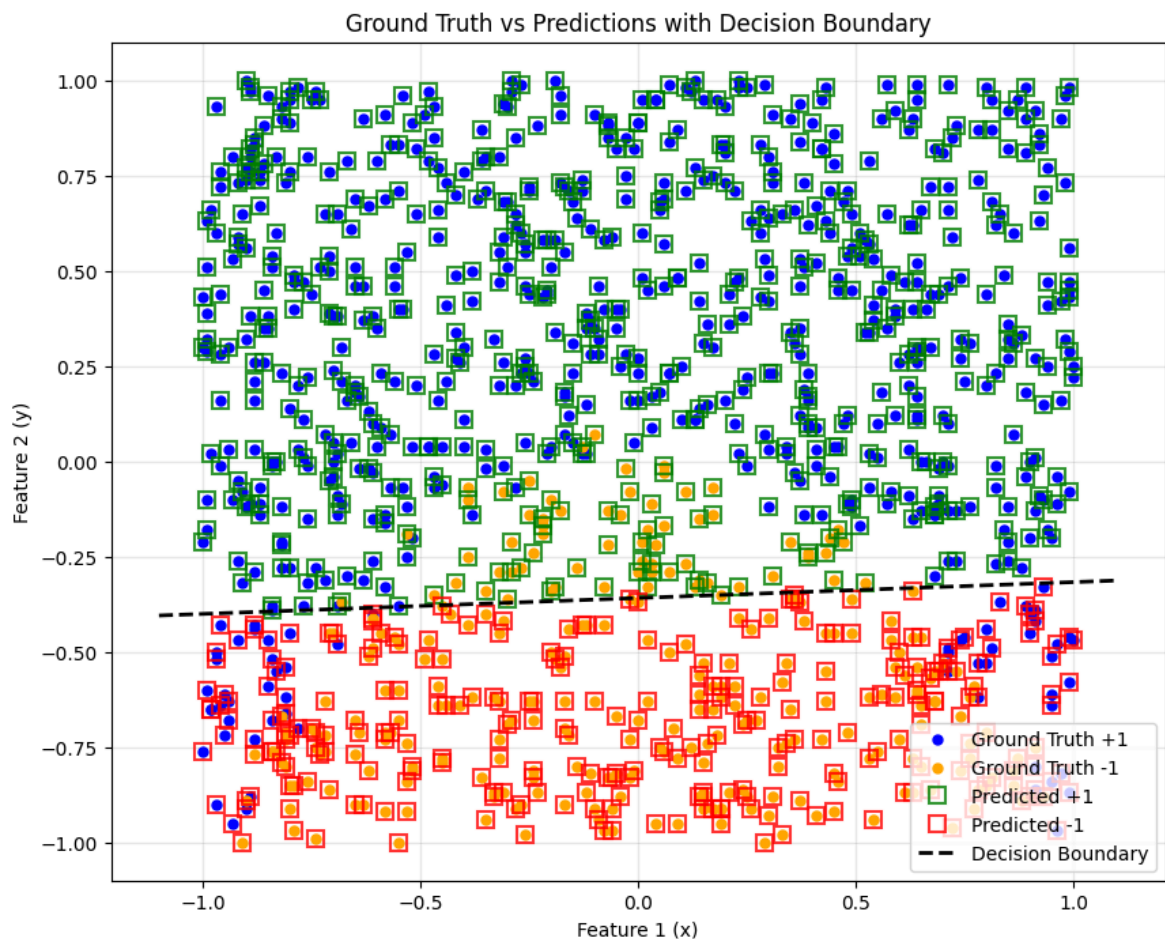
# Step 4: Create range of x1 values
x1_min = X[:, 0].min() - 0.1
x1_max = X[:, 0].max() + 0.1
x1_range = np.linspace(x1_min, x1_max, 100)

# Step 5: Calculate corresponding x2 values for decision boundary
x2_boundary = -(intercept + coef[0] * x1_range) / coef[1]

# Step 6: Plot decision boundary
plt.plot(x1_range, x2_boundary, 'k--', linewidth=2, label='Decision Boundary')

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('Ground Truth vs Predictions with Decision Boundary')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



iv) Briefly comment on how the predictions and the training data compare.

get some data to back up

```
In [10]: # Calculate additional metrics
accuracy = accuracy_score(y, y_pred)
precision = precision_score(y, y_pred)
recall = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print(f"\n=== Performance Metrics ===")
print(f"Accuracy:  {accuracy:.4f}")
print(f"Precision:  {precision:.4f}")
print(f"Recall:      {recall:.4f}")
print(f"F1-Score:    {f1:.4f}")
```

```
=== Performance Metrics ===
Accuracy:  0.8719
Precision: 0.9045
Recall:    0.9098
F1-Score:  0.9071
```

i) Train linear SVM classifiers for a wide range of values of the penalty parameter C e.g. $C = 0.001$, $C = 1$, $C = 100$. Give the SVM model for predictions and report

the parameter values of each trained model.

```
In [11]: from sklearn.svm import LinearSVC

# Train linear SVM classifiers for a wide range of C values
C_values = [0.0001, 0.001, 1, 100]
svm_models = {}
for C in C_values:
    svm = LinearSVC(C=C, max_iter=10000, dual=False)
    svm.fit(X, y)
    svm_models[C] = svm
    print(f"\nSVM model for C={C}:")
    print("  Intercept:", svm.intercept_)
    print("  Coefficients:", svm.coef_)

    y_pred = svm.predict(X)
```

```
SVM model for C=0.0001:
Intercept: [0.06219854]
Coefficients: [[-0.00364037  0.07394777]]
```

```
SVM model for C=0.001:
Intercept: [0.24237754]
Coefficients: [[-0.01796787  0.46712054]]
```

```
SVM model for C=1:
Intercept: [0.6448363]
Coefficients: [[-0.07336485  1.84353925]]
```

```
SVM model for C=100:
Intercept: [0.65218586]
Coefficients: [[-0.07433828  1.86233769]]
```

ii) Use each of these trained classifiers to predict the target values in the training data. Plot these predictions and the actual target values from the data, together with the classifier decision boundary.

```
In [71]: fig, axes = plt.subplots(2, 2, figsize=(14, 12), sharex=True, sharey=True)
axes = axes.flatten() # Flatten to make indexing easier

for idx, C in enumerate(C_values):
    ax = axes[idx]
    svm = svm_models[C]

    y_pred = svm.predict(X)

    # gt
    X_pos = X[y==1, :]
    X_neg = X[y==-1, :]

    # predictions
    X_pos_pred = X[y_pred==1, :]
    X_neg_pred = X[y_pred==-1, :]
```

```

# plot gt
ax.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', marker='.', color='red', alpha=0.8, fillstyle='none', markeredgewidth=1)
ax.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', marker='.', color='red', alpha=0.8, fillstyle='none', markeredgewidth=1)

# plot pred
ax.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 's', label='Predicted +1', color='green', alpha=0.8, fillstyle='none', markeredgewidth=1)
ax.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1', color='red', alpha=0.8, fillstyle='none', markeredgewidth=1)

# decision boundary
intercept = svm.intercept_[0]
coef = svm.coef_[0]

# create a range of x1 values for the decision boundary
x1_range = np.linspace(X[:, 0].min() - 0.1, X[:, 0].max() + 0.1, 100)
# calculate corresponding x2 values for the decision boundary
x2_boundary = -(intercept + coef[0] * x1_range) / coef[1]

# plot
ax.plot(x1_range, x2_boundary, 'k--', linewidth=2, label='Decision Boundary')

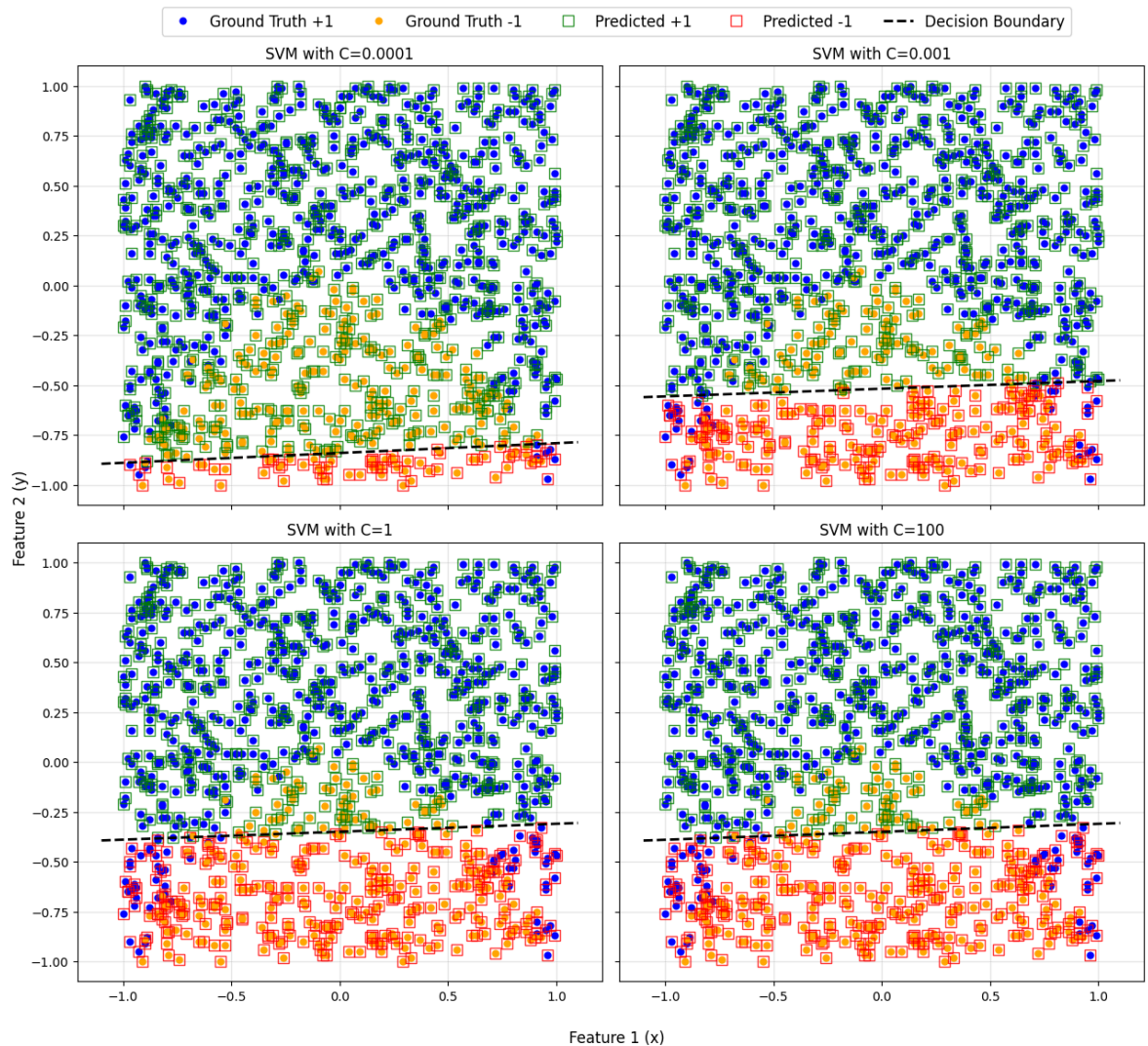
ax.set_title(f'SVM with C={C}')
ax.grid(True, alpha=0.3)

# shared axis
fig.supxlabel('Feature 1 (x)', fontsize=12)
fig.supylabel('Feature 2 (y)', fontsize=12)

# shared legend
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', bbox_to_anchor=(0.5, 0.98))

plt.tight_layout(rect=[0, 0.01, 0.93, 0.95])
plt.show()

```



```
In [ ]: print("="*100)
print("SVM MODEL PARAMETERS COMPARISON TABLE")
print("="*100)

print(f"{'C':<12} {'w1 (coef[0])':>15} {'w2 (coef[1])':>15} {'w0 (intercept)':>15}")
print("="*100)

# Extract parameters for each model
for C in C_values:
    svm = svm_models[C]

    # Get model parameters
    coef = svm.coef_[0]
    intercept = svm.intercept_[0]
    w_norm = np.linalg.norm(coef)
    margin = 2 / w_norm

    # Print row
    print(f"{'C':<12.4f} {'coef[0]':>15.6f} {'coef[1]':>15.6f} {'intercept':>15.6f}")

print("="*100)
```

SVM MODEL PARAMETERS COMPARISON TABLE

C	w1 (coef[0])	w2 (coef[1])	b (intercept)	w
Margin (2/ w)				
0.0001	-0.003640	0.073948	0.062199	0.074037
27.013404				
0.0010	-0.017968	0.467121	0.242378	0.467466
4.278386				
1.0000	-0.073365	1.843539	0.644836	1.844998
1.084012				
100.0000	-0.074338	1.862338	0.652186	1.863821
1.073065				

```
In [17]: # performance metrics for each SVM model
print("="*80)
print("SVM PERFORMANCE METRICS FOR DIFFERENT C VALUES")
print("="*80)

# Store metrics for comparison
metrics_table = []

for C in C_values:
    svm = svm_models[C]
    y_pred = svm.predict(X)

    # calc metrics
    accuracy = accuracy_score(y, y_pred)
    precision = precision_score(y, y_pred)
    recall = recall_score(y, y_pred)
    f1 = f1_score(y, y_pred)

    # confusion matrix
    cm = confusion_matrix(y, y_pred)
    TN, FP, FN, TP = cm.ravel()

    TPR = TP / (TP + FN)
    FPR = FP / (FP + TN)
    TNR = TN / (TN + FP)
    FNR = FN / (FN + TP)

    print(f"\n{'='*80}")
    print(f"SVM with C = {C}")
    print(f"{'='*80}")
    print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-Score:   {f1:.4f}")
    print(f"\nConfusion Matrix:")
    print(f"  TN: {TN:3d}  FP: {FP:3d}")
    print(f"  FN: {FN:3d}  TP: {TP:3d}")
    print(f"\nClassification Rates:")
    print(f"  TPR (Sensitivity): {TPR:.4f}")
    print(f"  TNR (Specificity): {TNR:.4f}")
```

```

print(f"  FPR: {FPR:.4f}")
print(f"  FNR: {FNR:.4f}")

metrics_table.append({
    'C': C,
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'TPR': TPR,
    'TNR': TNR
})

# print comparison table
print(f"\n{'='*80}")
print(f"COMPARISON TABLE")
print(f"{'='*80}")
print(f"{'C':<12} {'Accuracy':>10} {'Precision':>10} {'Recall':>10} {'F1-")
print(f"{'-'*80}")

for metrics in metrics_table:
    print(f"{'C':<12.4f} {'Accuracy':>10.4f} {'metrics['
          f"{'Recall':>10.4f} {'metrics['F1-Score']:>10.4f} {'metr

print(f"{'='*80}")

```

```

=====
=====
SVM PERFORMANCE METRICS FOR DIFFERENT C VALUES
=====
=====

=====
=====
SVM with C = 0.0001
=====
=====
Accuracy: 0.7427 (74.27%)
Precision: 0.7322
Recall: 0.9869
F1-Score: 0.8407

Confusion Matrix:
  TN: 64  FP: 248
  FN: 9   TP: 678

Classification Rates:
  TPR (Sensitivity): 0.9869
  TNR (Specificity): 0.2051
  FPR: 0.7949
  FNR: 0.0131

=====
=====
SVM with C = 0.001
=====
=====
Accuracy: 0.8519 (85.19%)
Precision: 0.8523
Recall: 0.9491
F1-Score: 0.8981

Confusion Matrix:
  TN: 199  FP: 113
  FN: 35   TP: 652

Classification Rates:
  TPR (Sensitivity): 0.9491
  TNR (Specificity): 0.6378
  FPR: 0.3622
  FNR: 0.0509

=====
=====
SVM with C = 1
=====
=====
Accuracy: 0.8699 (86.99%)
Precision: 0.9054
Recall: 0.9054
F1-Score: 0.9054

Confusion Matrix:
  TN: 247  FP: 65
  FN: 65   TP: 622

```

Classification Rates:
 TPR (Sensitivity): 0.9054
 TNR (Specificity): 0.7917
 FPR: 0.2083
 FNR: 0.0946

SVM with C = 100

Accuracy: 0.8699 (86.99%)
 Precision: 0.9054
 Recall: 0.9054
 F1-Score: 0.9054

Confusion Matrix:
 TN: 247 FP: 65
 FN: 65 TP: 622

Classification Rates:
 TPR (Sensitivity): 0.9054
 TNR (Specificity): 0.7917
 FPR: 0.2083
 FNR: 0.0946

COMPARISON TABLE

C	Accuracy	Precision	Recall	F1-Score	TPR	TNR
0.0001	0.7427	0.7322	0.9869	0.8407	0.9869	0.7427
0.0010	0.8519	0.8523	0.9491	0.8981	0.9491	0.7917
0.0100	0.8699	0.9054	0.9054	0.9054	0.9054	0.7917
0.1000	0.8699	0.9054	0.9054	0.9054	0.9054	0.7917
1.0000	0.8699	0.9054	0.9054	0.9054	0.9054	0.7917
10.0000	0.8699	0.9054	0.9054	0.9054	0.9054	0.7917
100.0000	0.8699	0.9054	0.9054	0.9054	0.9054	0.7917

iii) What is the impact on the model parameters of changing C, and why ? What is the impact on the SVM predictions?

Practical Implications C too small: Underfitting - model is too simple C too large: Overfitting - model memorizes training data Optimal C: Found through cross-validation, balances bias and variance

iv) How do the SVM model parameters and predictions compare to those of the logistic regression model in part (a)?

Part C

i) Now create two additional features by adding the square of each feature (i.e. giving four features in total). Train a logistic regression classifier. Give the model and the trained parameter values.

```
In [18]: # polynomial features (original + squared)
X_poly = np.column_stack((X1, X2, X1**2, X2**2))

print("Original features shape:", X.shape)
print("Polynomial features shape:", X_poly.shape)
print("Feature names: [X1, X2, X12, X22]")
print(X_poly[:,2])
```

```
Original features shape: (999, 2)
Polynomial features shape: (999, 4)
Feature names: [X1, X2, X12, X22]
[[ 6.400e-01  3.000e-01  4.096e-01  9.000e-02]
 [-1.000e-02  8.200e-01  1.000e-04  6.724e-01]]
```

```
In [19]: clf_poly = LogisticRegression()
clf_poly.fit(X_poly, y)
```

```
Out[19]: ▼ LogisticRegression ⓘ ?
          ► Parameters
```

```
In [20]: print("=== Logistic Regression with Polynomial Features ===")
print("Intercept (bias):", clf_poly.intercept_[0])
print("Coefficients:", clf_poly.coef_[0])
print("\nFeature coefficients:")
print(f"X1 (x) coefficient: {clf_poly.coef_[0][0]:.4f}")
print(f"X2 (y) coefficient: {clf_poly.coef_[0][1]:.4f}")
print(f"X12 (x2) coefficient: {clf_poly.coef_[0][2]:.4f}")
print(f"X22 (y2) coefficient: {clf_poly.coef_[0][3]:.4f}")
```

=== Logistic Regression with Polynomial Features ===

Intercept (bias): 0.2984504002925016

Coefficients: [-0.15002377 6.96714254 6.44457421 -0.54193648]

Feature coefficients:

X1 (x) coefficient: -0.1500

X2 (y) coefficient: 6.9671

X1² (x²) coefficient: 6.4446

X2² (y²) coefficient: -0.5419

ii) Use the trained classifier to predict the target values in the training data. Plot these predictions and the actual target values from the data using the same style of plot as before i.e. using just the two original features as x and y axes. Compare and discuss. How do the predictions compare with those in parts (a) and (b) above?

```
In [22]: y_pred_poly = clf_poly.predict(X_poly)
y_pred_proba_poly = clf_poly.predict_proba(X_poly)

print("Polynomial LR Predictions:", y_pred_poly[:10])
print("Polynomial LR Prediction probabilities:", y_pred_proba_poly[:10])

plt.figure(figsize=(10, 8))

X_pos = X[y==1, :]
X_neg = X[y==-1, :]

X_pos_pred = X[y_pred_poly==1, :]
X_neg_pred = X[y_pred_poly==-1, :]

# ground truth
plt.plot(X_pos[:, 0], X_pos[:, 1], '.', label='Ground Truth +1', markersize=10)
plt.plot(X_neg[:, 0], X_neg[:, 1], '.', label='Ground Truth -1', markersize=10)

# predictions
plt.plot(X_pos_pred[:, 0], X_pos_pred[:, 1], 'o', label='Predicted +1', markerfacecolor='lightblue', alpha=0.8, fillstyle='none', markeredgewidth=2)
plt.plot(X_neg_pred[:, 0], X_neg_pred[:, 1], 's', label='Predicted -1', markerfacecolor='orange', alpha=0.8, fillstyle='none', markeredgewidth=2)

intercept = clf_poly.intercept_[0]
coef = clf_poly.coef_[0]

print(f"Decision boundary equation:")
print(f"{coef[3]:.4f}*X22 + {coef[1]:.4f}*X2 + ({intercept:.4f} + {coef[0]:.4f}*X1)")

# Create range of X1 values
x1_range = np.linspace(X[:, 0].min() - 0.3, X[:, 0].max() + 0.3, 200)

# For each X1, solve quadratic equation for X2
a = coef[3]
b = coef[1]
c_values = intercept + coef[0]*x1_range + coef[2]*x1_range**2
```

```

# Quadratic formula:  $X_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 
discriminant = b**2 - 4*a*c_values

# Only plot where discriminant >= 0 (real solutions)
valid_idx = discriminant >= 0
x1_valid = x1_range[valid_idx]
disc_valid = discriminant[valid_idx]

# Two solutions (+ and -)
x2_plus = (-b + np.sqrt(disc_valid)) / (2*a)
x2_minus = (-b - np.sqrt(disc_valid)) / (2*a)

# Plot both branches of the boundary
plt.plot(x1_valid, x2_plus, 'k--', linewidth=2, label='Decision Boundary')

# Wrong plot
# plt.plot(x1_valid, x2_minus, 'k--', linewidth=2)

plt.xlabel('Feature 1 (x)')
plt.ylabel('Feature 2 (y)')
plt.title('Polynomial Logistic Regression: Ground Truth vs Predictions')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

Polynomial LR Predictions: [1 1 1 1 1 1 1 1 -1 1]

Polynomial LR Prediction probabilities: [[7.51358011e-03 9.92486420e-01]

[3.50779427e-03 9.96492206e-01]

[1.78139428e-03 9.98218606e-01]

[1.85422190e-03 9.98145778e-01]

[2.41063081e-02 9.75893692e-01]

[8.18078262e-03 9.91819217e-01]

[2.22605298e-01 7.77394702e-01]

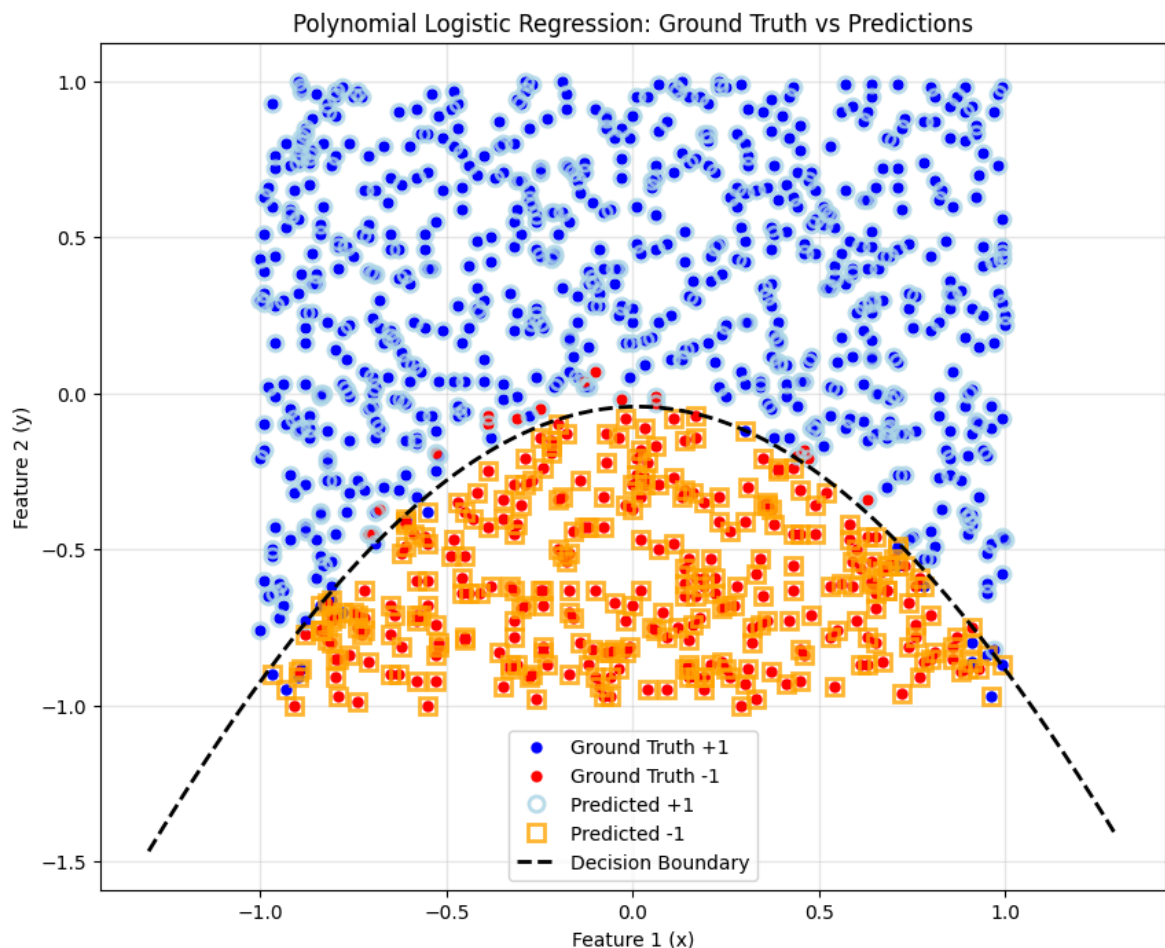
[5.60437790e-05 9.99943956e-01]

[9.85606718e-01 1.43932816e-02]

[9.29162101e-03 9.90708379e-01]]

Decision boundary equation:

$$-0.5419 \cdot X_2^2 + 6.9671 \cdot X_2 + (0.2985 + -0.1500 \cdot X_1 + 6.4446 \cdot X_1^2) = 0$$



In [23]: `from sklearn.metrics import accuracy_score, precision_score, recall_score`

```
# baseline
unique_classes, counts = np.unique(y, return_counts=True)
most_common_class = unique_classes[np.argmax(counts)]

print(f"Most common class in dataset: {most_common_class}")
print(f"Class distribution:")
for cls, count in zip(unique_classes, counts):
    print(f"  Class {cls:>2}: {count} samples ({count/len(y)*100:.1f}%)")

y_pred_baseline = np.full_like(y, most_common_class)

# predict
y_pred_poly = clf_poly.predict(X_poly)

# from previous
def calculate_metrics(y_true, y_pred, model_name):
    """Calculate and return all performance metrics"""

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    # Confusion matrix
    # Possibly not needed but was used in biometrics course
    cm = confusion_matrix(y_true, y_pred)
    TN, FP, FN, TP = cm.ravel()
```

```

# Rates
TPR = TP / (TP + FN)
FPR = FP / (FP + TN)
TNR = TN / (TN + FP)
FNR = FN / (FN + TP)

print(f"\n{'='*50}")
print(f"{model_name}")
print(f"{'='*50}")
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"\nConfusion Matrix:")
print(f"  TN: {TN:3d}  FP: {FP:3d}")
print(f"  FN: {FN:3d}  TP: {TP:3d}")
print(f"\nRates:")
print(f"  TPR (Sensitivity): {TPR:.4f}")
print(f"  TNR (Specificity): {TNR:.4f}")
print(f"  FPR: {FPR:.4f}")
print(f"  FNR: {FNR:.4f}")

return {
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1,
    'TPR': TPR,
    'TNR': TNR,
    'FPR': FPR,
    'FNR': FNR
}

baseline_metrics = calculate_metrics(y, y_pred_baseline, "BASELINE (Always)")
poly_lr_metrics = calculate_metrics(y, y_pred_poly, "POLYNOMIAL LOGISTIC")

print(f"\n{'='*70}")
print(f"PERFORMANCE COMPARISON")
print(f"{'='*70}")
print(f"{'Metric':<20} {'Baseline':>15} {'Poly LR':>15} {'Improvement':>15}")
print(f"{'-'*70}")

for metric_name in ['accuracy', 'precision', 'recall', 'f1']:
    baseline_val = baseline_metrics[metric_name]
    poly_val = poly_lr_metrics[metric_name]
    improvement = poly_val - baseline_val

    print(f"{metric_name.capitalize():<20} {baseline_val:>15.4f} {poly_val:>15.4f} {improvement:>15.4f}")

print(f"{'='*70}")

```

Most common class in dataset: 1

Class distribution:

Class -1: 312 samples (31.2%)

Class 1: 687 samples (68.8%)

=====

BASELINE (Always Predict Most Common Class)

=====

Accuracy: 0.6877 (68.77%)

Precision: 0.6877

Recall: 1.0000

F1-Score: 0.8149

Confusion Matrix:

TN: 0 FP: 312

FN: 0 TP: 687

Rates:

TPR (Sensitivity): 1.0000

TNR (Specificity): 0.0000

FPR: 1.0000

FNR: 0.0000

=====

POLYNOMIAL LOGISTIC REGRESSION

=====

Accuracy: 0.9640 (96.40%)

Precision: 0.9752

Recall: 0.9723

F1-Score: 0.9738

Confusion Matrix:

TN: 295 FP: 17

FN: 19 TP: 668

Rates:

TPR (Sensitivity): 0.9723

TNR (Specificity): 0.9455

FPR: 0.0545

FNR: 0.0277

=====

PERFORMANCE COMPARISON

=====

Metric	Baseline	Poly LR	Improvement
Accuracy	0.6877	0.9640	+0.2763
Precision	0.6877	0.9752	+0.2875
Recall	1.0000	0.9723	-0.0277
F1	0.8149	0.9738	+0.1588

=====